Logic Meets Language: Hybrid Query Answering over Incomplete Text-Labeled Graphs

Anonymous ACL submission

Abstract

Text-labeled graph databases combine symbolic graph structures with rich textual descriptions, enabling expressive representations for real-world data. However, answering queries over such databases remains challenging when queries require both precise logical constraints and flexible semantic text interpretation, especially under information incompleteness. We introduce the task of hybrid query answering (hybrid QA) over *incomplete text-labeled graph databases*, which requires joint reasoning over both symbolic and textual knowledge under incompleteness. To investigate this problem, we formalize a query language that integrates logical constraints with free-form textual descriptions and construct two new benchmark datasets from biological and e-commerce domains. We propose HybridQE, a hybrid query embedding framework that aligns with our query language via embedding and combining the symbolic and textual constraints in a unified embedding space. Experiments on the proposed benchmarks show that existing methods struggle with this setting, while HybridQE provides consistent improvements across diverse hybrid query types, highlighting both the difficulty of HybridQA and the promise of hybrid query embeddings for reasoning under incompleteness.

1 Introduction

Text-labeled graph databases (He and McAuley, 2016; Jin et al., 2023; Wu et al., 2025) combine textual descriptions with graph structured data, facilitating a flexible and expressive framework for information storage (Yan et al., 2023; Kuang et al., 2023). Effective information retrieval systems over text-labeled graph databases are crucial for various downstream tasks, such as text classification (Zhao et al., 2023) and personalized recommendation (Zhu et al., 2021).

A body of work integrates textual and relational evidence for question answering. GraftNet (Sun et al., 2018) and PullNet (Sun et al., 2019) retrieve and reason jointly over text passages and knowledge graph triples for multi-hop natural language questions. More recently, Wu et al. (2025) introduce STaRK, a benchmark for LLM-based retrieval over semi-structured knowledge bases that pair textual entity descriptions with relational graph structure. To address this setting, Retrieval-Augmented Generation (RAG) and Graph RAG methods (Peng et al., 2024; Sun et al., 2024; Yasunaga et al., 2021a) are employed to retrieve textual and relational information, respectively. When questions require evidence from both modalities simultaneously, however, methods relying on a single retrieval channel fail to capture the full evidence. Lee et al. (2025) address this with HybGRAG, a hybrid agentic retrieval framework that jointly queries text documents and graph relations.

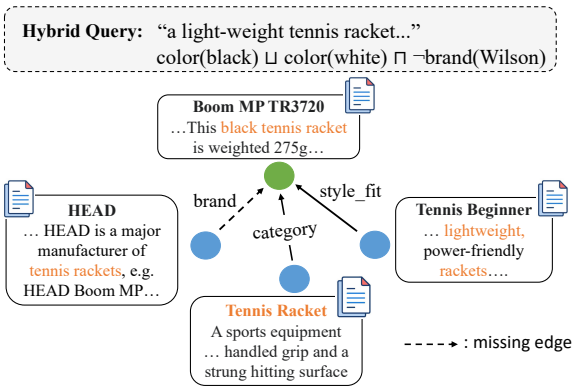


Figure 1: An illustration of a hybrid QA system over incomplete text-labeled graphs in the e-commerce setting.

Despite this progress, two critical issues remain unaddressed in existing work. The first is the inability to simultaneously enforce *precise logical constraints* and capture *nuanced textual semantics* within a single query. Consider the e-commerce query example: "*black or white tennis racket that is not Wilson and very lightweight.*" It illustrates two common phenomena in real-world queries. First, semantic flexibility: the attribute "*very lightweight*"

Method	Logical	Semantic	Incomplet.
BM25	×	✓	×
ANCE	×	✓	×
ToG	×	✓	×
HybGRAG	×	✓	×
BetaE	✓	×	✓
GammaE	✓	×	✓
HybridQE	✓	✓	✓

Table 1: Qualitative comparison of baseline methods for different types of retrieval and query answering tasks.

conveys a nuanced, context-dependent concept that cannot be directly mapped to a discrete symbolic attribute. Second, logical ambiguity: the query allows multiple different interpretations, such as "*very lightweight tennis rackets that are not Wilson*" or "*normal or even heavy-weighted tennis rackets that are not Wilson*". Therefore, a hybrid query framework consisting of symbolic logical constraints and a textual description of the searched result is essential to bridge this gap. Under such a framework, the above query can be reformulated as, $\exists \mathsf{color}.(\{\mathsf{black}\} \sqcup \{\mathsf{white}\}) \sqcap \neg \exists \mathsf{brand}(\{\mathsf{Wilson}\}) \sqcap \{$"lightweight tennis racket"$\}$, preserving both logical precision and semantic flexibility.

The second is the failure to account for *missing symbolic data*. Existing retrieval frameworks assume complete and reliable symbolic descriptions for the graph entities, whereas real-world annotations are often sparse, noisy, or outdated due to the limits of manual curation (Xu et al., 2019a,b). As shown in Fig. 1, answering queries over text-labeled graphs frequently requires inferring the missing attributes from the item's descriptions and relational structures. Recent work on query-embedding (QE) methods (Ren and Leskovec, 2020; Yang et al., 2022) map symbolic knowledge and queries into a vector space, but only assume symbolic queries. LLMs are employed to complete such missing information (Xu et al., 2024), however their effectiveness is limited in domains that are uncovered by the training corpus of the LLMs.

Our contributions are as follows: We formalize a hybrid query language with logical operators over both symbolic and textual modalities (§2); construct two new benchmark datasets satisfying incompleteness assumptions on text-labeled graphs (§3); propose HybridQE, a hybrid query embedding framework that jointly reasons over hybrid queries under incompleteness (§4, Table 1); and present extensive experiments addressing four research questions on the effectiveness, robustness, and adaptability of HybridQE(§5).

2 Problem Statement

In this paper we study the hybrid query answering problem, which is an extension of the query answering problem over incomplete knowledge graphs (He et al., 2024) to include text descriptions in the data and in the queries. To formalize this problem, we next define how we extend knowledge graphs and how we extend the query language.

2.1 Text-labeled Knowledge Graphs

Knowledge graph query answering has been studied over structures $G = (V, R, E)$ where V and R are two finite disjoint sets of *nodes* and *relation names*, and $E \subseteq V \times R \times V$ is a set of labeled edges. *Text-labeled knowledge graphs* extend these structures with textual information for nodes and edges. For this extension, we assume the existence of a set T, disjoint from V and R containing all possible textual descriptions.

Definition 1 (Text-labeled knowledge graph)**.** *A text-labeled knowledge graph G is a tuple (V, R, E, T, τ), where (V, R, E) is a knowledge graph, and $\tau : V \cup R \to T$ is a function mapping nodes and relation names to text.*

2.2 Hybrid Query Language

We refer to the QA problem over text-labeled knowledge graphs as the hybrid query answering problem (HybridQA). The name *hybrid* comes from the combination of text and structure. To formally define this problem, we need first to define the query language. To this end we build on top of the $\mathcal{SROI}^-$ description logics formulation of query answering by He et al. (He et al., 2024).[1]

Definition 2 (Hybrid queries)**.** Hybrid queries Q over text-labeled knowledge graphs *are defined by the following grammar:*

$$Q ::= \top \mid \{v\} \mid \{t\} \mid \neg Q \mid Q \sqcap Q \mid \exists R.Q\,,$$
$$R ::= r \mid r^- \mid t \mid t^-\,,$$

where $\top$ is the universal concept, t, v, and r stand for elements in sets T, V, and R, respectively.

The semantics are defined as usual for all elements, except expressions of the form $\{t\}$, called *text nominals*, which match entities whose textual

[1] See Appendix A for the $\mathcal{SROI}^-$ syntax and semantics.

description is semantically similar to t, and expressions of the form $\exists t.Q$ and $\exists t^{-}.Q$, called *textual relational existentials*, where t is interpreted as a relation name. Standard node nominals $\{v\}$ match exactly the individual node $v \in V$.

2.3 Hybrid QA under Incompleteness

At inference time, the input of the HybridQA problem consists of a hybrid query Q and an incomplete text-labeled knowledge graph $G_{\text{train}} = (V, R, E_{\text{train}}, T, \tau)$. A HybridQA model produces a score for every node v $\in$ V, representing how likely v is to satisfy the query Q.

Following the standard query answering setting over incomplete knowledge graphs, we assume that the observed graph G_{train} is a subgraph of a complete graph $G_{\text{test}} = (V, R, E_{\text{train}} \cup E_{\text{missing}}, T, \tau)$, where the set E_{missing} contains edges that are not available during training, and $E_{\text{missing}} \cap E_{\text{train}} = \emptyset$.

For a query Q, let $Q^{G_{\text{train}}}$ and $Q^{G_{\text{test}}}$ denote the sets of answers obtained by evaluating the query over the training graph and the complete graph, respectively. We distinguish two answer sets: the *easy answer set* $A_{\text{easy}}(Q) = Q^{G_{\text{train}}}$, containing answers directly retrievable from the observed graph, and the *hard answer set* $A_{\text{hard}}(Q) = Q^{G_{\text{complete}}} \setminus Q^{G_{\text{train}}}$, containing answers that require inference over missing edges. Our evaluation focuses on $A_{\text{hard}}(Q)$, as these answers test the model's ability to reason under incompleteness.

Given a hybrid query Q, the goal of HybridQA is to rank entities such that nodes in $A_{\text{hard}}(Q)$ receive higher scores than non-answers.

3 HybridQA Benchmarks

We introduce two benchmarks for the HybridQA problem, ProdHybridQA and BioHybridQA, which are derived from real-world e-commerce and protein-disease data (Zhang et al., 2022; Xu et al., 2019b; Wu et al., 2025). The existing text-labeled graph QA datasets are not suitable to evaluate the HybridQA problem. Indeed, some (Wu et al., 2025) assume completeness and support only textual queries, while symbolic QA (Ren et al., 2019; Ren and Leskovec, 2020; Gregucci et al., 2025) benchmarks do not include text descriptions in the graph. Our new datasets introduce two key innovations: incomplete text-labeled knowledge graphs and hybrid queries for these graphs.

Hybrid Query Formulation. Our proposed dataset queries adhere to established query structures commonly used in symbolic QA tasks, i.e. 1p, 2p, 3p, 2i, 3i, 1n, 2in, 3in, 2u, and 3u.[2] Figure 2 illustrates the structures of these symbolic queries. We make a conjunction of each of these queries Q with a text nominal $\{t\}$ to create a hybrid query of the form $Q \sqcap \{t\}$. Table 2 shows examples queries for the ProdHybridQA dataset.

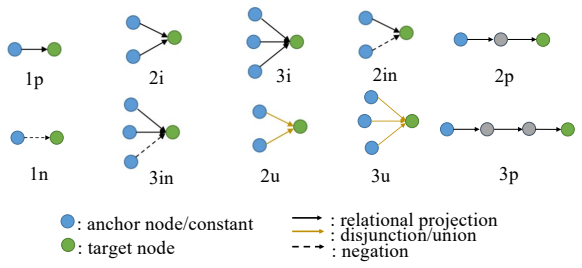


Figure 2: Query structures of our datasets queries. Anchor nodes (in cyan) and relations names are to be specified to instantiate the queries. Each structure is named using abbreviations: "p" for projection, "i" for intersection, "u" for union, and "n" for negation.

Table 2: Example queries in ProdHybridQA

Type	Hybrid Query Example
1p	"Could you find a body deodorant made in powder form that has a gentle floral fragrance?" $\sqcap$ $\exists$brand.{HoneybeeGardens}
2i	"Could you provide a battery that is in black color?" $\sqcap$ $\exists$binding.{ConsElect} $\sqcap$ $\exists$brand.{Neewer}
3i	"Could you find a candle in a jar container that has a single wick?" $\sqcap$ $\exists$category.{candle} $\sqcap$ $\exists$scent.{SweetPea} $\sqcap$ $\exists$binding.{miscellaneous}
1n	"Could you find breakfast cereal produced by the brand KIND?" $\sqcap$ $\neg\exists$flavor.{oat}
2in	"Could you find a candle made from a soy-blend wax material?" $\sqcap$ $\exists$container_type.{jar} $\sqcap$ $\neg\exists$manufacturer.{MVPGroup})
3in	"Could you provide a cake that comes packaged in a box?" $\sqcap$ $\neg\exists$manufacturer.{SweetSams} $\sqcap$ $\exists$binding.{grocery} $\sqcap$ $\exists$flavor.{fruit}
2u	"Could you find a blood pressure monitor that comes in a gray color?" $\sqcap$ ($\exists$brand.{Beurer} $\sqcup$ $\exists$voltage_units.{volts})
3u	"Could you find an oil-based body lubricant with a ylang fragrance that has its shelf life specified in days?" $\sqcap$ ($\exists$brand.{PipedreamProducts} $\sqcup$ $\exists$brand.{FleurDeSpa} $\sqcup$ $\exists$brand.{OPO})

Incompleteness Conditions Our datasets introduce two complementary forms of incompleteness to ensure that hybrid query answering requires gen-

[2]Note that ProdHybridQA is built on a hyperedge graph, where product–attribute relationships are encoded as hyperedges rather than binary triples. Multi-hop projection queries (2p, 3p) are not defined for this dataset, since hyperedges do not compose transitively across intermediate nodes.

uine inference rather than simple lookup.

On the symbolic side, to be certain about each test query answer, we require two types of edges, edges that are at hand (i.e., in set E_{train}) and missing edges (i.e., in set E_{missing}). This controlled symbolic incompleteness is aligned with the QA setting. On the text side, real-world textual descriptions have a high variability, noise, and partial information coverage. In our generated queries, we simulated this incompleteness by generating textual descriptions from a randomly sampled subset of triples associated with the target entity. As a result, the query textual descriptions capture fragmentary information, requiring models to infer missing data from both the text descriptions and the incomplete graph structure.

We refer the reader to Appendix B for full details on the query generation process and a systematic justification of how the generation procedure fulfills our dataset assumptions. Dataset statistics are provided in Table 5.

4 HybridQE for HybridQA

We next present the rational, the query embedding design, and the training aspects of HybridQE.

4.1 Rationale

We hypothetize that a method for the HybridQA problem would benefit from encoding the symbolic and the textual parts directly, that is, without transforming one into the other. Translating symbolic queries into natural-language questions would inevitably lead to loss or distortion of the original logical meaning of operators such as conjunction, disjunction, and negation (Yih et al., 2016; Banerjee, 2023). Conversely, converting natural language questions into symbolic queries through semantic parsing also struggles to capture the nuances, often yielding incorrect or incomplete logical forms (Feng and He, 2025; Zhao et al., 2025; Shen et al., 2020).

Additionally, HybridQE is expected to support joint reasoning over hybrid queries in the presence of incompleteness. Missing information in the graph may be recoverable through either the associated textual descriptions or latent structural patterns in the database. We hypothesize that a robust hybrid QA model should therefore enable both text-driven and structure-driven completion, allowing these complementary inference pathways to interact during reasoning.

4.2 Hybrid Query Embedding

Our method, HybridQE, infers query answers by embedding both entities and hybrid queries into a shared vector space $\mathbf{E}^d$ and ranking entities by their distance to the query embedding. Entities positioned closer are predicted as more plausible answers. Text descriptions and relations are embedded to spaces $\mathbf{T}^d$ and $\mathbf{R}^d$, and then mapped to space $\mathbf{E}^d$ when representing queries and entities.[3]

Entity Embeddings with Query-Conditioned Similarity. We next define how HybridQE embeds entities into the vector space $\mathbf{E}^d$. Unlike prior methods (Yang et al., 2022; Ren and Leskovec, 2020), where an entity ee e is assigned a single static vector, HybridQE produces a query-conditioned entity embedding $\boldsymbol{e}_Q \in \mathbf{E}^d$ that selectively emphasizes the portions of the entity's text description most relevant to query Q.

Let $\xi : T \to \mathbf{T}^d$ denote a pretrained language model that maps text to vectors. For every entity $e \in V$ with text description $t_e = \tau(e)$, we obtain a token-level representation $\mathbf{E} = \xi_{tok}(t_e) \in \mathbb{R}^{n \times d}$ where n is the number of tokens in t_e and d is the hidden dimension. We also compute a pooled entity representation $\bar{\boldsymbol{e}} = \text{mean-pool}(\mathbf{E}) \in \mathbb{R}^d$. For a hybrid query Q of the form $\{t\} \sqcap \phi$, we define two query-side vectors, $\boldsymbol{q}_t = \xi(t)$ and $\boldsymbol{q}_\phi = \xi(t_\phi)$, where t_ϕ is the text description resulting from verbalizing query ϕ into natural language.

To identify which tokens of the entity description are most relevant to each query component, we compute attention scores by taking the dot product of each token embedding with the corresponding query vector:

$$\boldsymbol{\alpha}_t = \text{softmax}(\mathbf{E}\boldsymbol{q}_t) \in \mathbb{R}^n, \quad (1)$$

$$\boldsymbol{\alpha}_\phi = \text{softmax}(\mathbf{E}\boldsymbol{q}_\phi) \in \mathbb{R}^n. \quad (2)$$

where $\mathbf{E}\boldsymbol{q}_t$ is the vector of dot-product scores between each of the n token embeddings and the query vector. The softmax normalizes these into a probability distribution over tokens. The resulting query-conditioned context vectors are:

$$\tilde{\mathbf{a}}_t = \mathbf{E}^\top \boldsymbol{\alpha}_t \in \mathbb{R}^d \quad (3)$$

$$\tilde{\mathbf{a}}_\phi = \mathbf{E}^\top \boldsymbol{\alpha}_\phi \in \mathbb{R}^d \quad (4)$$

representing a d-dimensional summary of the entity description weighted by relevance to each query

[3] We use different notations for the three spaces because our method is model-agnostic. In some models the three spaces are the same, $\mathbb{R}^d$, but this is not generally true.

component. To modulate how much each query component should influence the final entity representation, we apply learnable gating functions:

$$\boldsymbol{g}_t = \sigma\left(\boldsymbol{W}_t\left[\bar{\boldsymbol{e}}; \tilde{\mathbf{a}}_t\right]\right), \tag{5}$$

$$\boldsymbol{g}_\phi = \sigma\left(\boldsymbol{W}_\phi\left[\bar{\boldsymbol{e}}; \tilde{\mathbf{a}}_\phi\right]\right), \tag{6}$$

where $[\cdot;\cdot]$ denotes concatenation, σ is the sigmoid activation function, $\boldsymbol{W}_t$ and $\boldsymbol{W}_\phi$ are learnable weights and the gates $\boldsymbol{g}_t$ and $\boldsymbol{g}_t$ dynamically adjust the contribution of each modality. The final query-specific entity embedding is produced by combining these gated contributions and projecting them into the shared embedding space via a Multi-Layer Perceptron (MLP) $\mathbf{P}_{\mathrm{E}} : \mathbf{T}^d \to \mathbf{E}^d$:

$$\boldsymbol{e}_Q = \mathbf{P}_{\mathrm{E}}(\bar{\boldsymbol{e}} + \boldsymbol{g}_t \odot \tilde{\mathbf{a}}_t + \boldsymbol{g}_\phi \odot \tilde{\mathbf{a}}_\phi), \tag{7}$$

where $\odot$ is the Hadamard product. This design ensures that entity embeddings are not static database artifacts but contextualized representations that reflect both what the query is asking and how the entity's description relate to it.

Relation Embeddings. We compute the embeddings of relations by mapping the text description of each relation name $r \in R$ and its inverse r^- into the embedding space $\mathbf{R}^d$ by using their textual descriptions:

$$\boldsymbol{r} = \mathbf{P}_{\mathrm{R}}(\xi(t_r)), \quad \boldsymbol{r}^- = \mathbf{P}_{\mathrm{R}}(\xi(t_{r^-})), \tag{8}$$

where $\mathbf{P}_{\mathrm{R}} : \mathbf{T}^d \to \mathbf{R}^d$ is a learnable projection layer, t_r and t_{r^-} are the text descriptions of relation names r and r^-.

Symbolic Query Encoder. On top of the embedding of entities and relations, we define a symbolic query encoder, $\mathbf{Emb}_\phi$. The goal is to preserve the exact logical structure of ϕ while embedding it into the shared vector space $\mathbf{E}^d$ where reasoning can be performed jointly with textual information. Unlike natural-language reformulations of symbolic queries, which risk losing their exact logical operators, our encoder preserves the logical precision by directly modeling the symbolic form using neural operators defined over the embedding space. The symbolic encoder contributes explicit logical inference capabilities inherited from neural query embedding models, which have been shown to reason over unseen patterns and missing edges in incomplete graphs (He et al., 2024). The signatures of these neural operators are the following: $\mathrm{Nominal} : \mathbf{E}^d \to \mathbf{E}^d$, $\mathrm{RelT} : \mathbf{E}^d \times \mathbf{R}^d \to \mathbf{E}^d$, $\mathrm{Intersect} : \mathbf{E}^d \times \ldots \times \mathbf{E}^d \to \mathbf{E}^d$, $\mathrm{Union} : \mathbf{E}^d \times \ldots \times \mathbf{E}^d \to \mathbf{E}^d$ and $\mathrm{Complement} : \mathbf{E}^d \to \mathbf{E}^d$. These neural operators define recursively the query embedding of a query Q as follows:

$$\begin{aligned}
\mathbf{Emb}_{\{t\}} &::= \mathrm{Nominal}(\mathbf{P}_{\mathrm{E}}(\xi(t))), \\
\mathbf{Emb}_{\{e\}} &::= \mathrm{Nominal}(\boldsymbol{e}_Q), \\
\mathbf{Emb}_{\exists r.\phi} &::= \mathrm{RelT}_r(\mathbf{Emb}_\phi, \boldsymbol{r}), \\
\mathbf{Emb}_{\phi_1 \sqcap \ldots \sqcap \phi_n} &::= \mathrm{Intersect}(\mathbf{Emb}_{\phi_1}, \ldots, \mathbf{Emb}_{\phi_n}), \\
\mathbf{Emb}_{\phi_1 \sqcup \ldots \sqcup \phi_n} &::= \mathrm{Union}(\mathbf{Emb}_{\phi_1}, \ldots, \mathbf{Emb}_{\phi_n}), \\
\mathbf{Emb}_{\neg\phi} &::= \mathrm{Complement}(\mathbf{Emb}_\phi),
\end{aligned} \tag{9}$$

HybridQE is model-agnostic with respect to the exact implementation of these operators and can operate with different query embedding backbones. Instantiations for BetaE (Ren and Leskovec, 2020) and GammaE (Yang et al., 2022) are provided in Appendix C.

The layer $\mathbf{P}_{\mathrm{E}}$ projects text descriptions into the same latent space as symbolic queries with the aim to supports bidirectional information flow and joint inference across modalities. Indeed, hybrid queries Q of the form $\{t\} \sqcap \phi$ that combine a textual description with a symbolic formula are particular case of the conjunction, $\mathbf{Emb}_Q = \mathrm{Intersect}(\mathbf{Emb}_{\{t\}}, \mathbf{Emb}_\phi)$. We hypothesize that this design keeps both modalities isolated during encoding to preserve the semantic precision and flexibility in hybrid query, while the shared embedding space enables the two modalities to exchange information, allowing textual evidence to fill missing symbolic relations and symbolic constraints to filter irrelevant textual information. As a result, HybridQE preserves the exact logical structure of the symbolic query while extending neural reasoning to free-text semantics and incomplete data.

4.3 Model Training

Learning Objective. Given a hybrid query Q and a text-labeled graph G, the goal of query embedding approaches is to maximize the predictions of *positive answers* to query Q (i.e., answer entity a that are in Q^G and minimize the prediction of *negative answers* to query Q (i.e., entities b that are in $\neg Q^G$). In the representation space, the evaluation of how likely an entity is an answer to a query is computed with a distance function with signature $\mathrm{Dist} : \mathbf{E}^d \times \mathbf{E}^d \to \mathbb{R}$. Given an entity e and a query Q, and number γ denoting a margin hyperparameter, we abbreviate $\log \sigma(\gamma - \mathrm{Dist}(\mathbf{Emb}_Q, \mathbf{Emb}_{\{b\}}))$ as Δ_Q^e. Then, given a query Q the goal of query embedding is to

Table 3: The MRR performance of HybridQE and baseline models on ProdHybridQA and BioHybridQA benchmark datasets. The results of our methods are highlighted.

Dataset	Model	Avg	1p	2i	3i	2u	3u	1n	2in	3in	2p	3p
ProdHybridQA	BM25	9.23	9.45	12.74	11.96	13.32	11.6	**6.99**	4.55	3.23	-	-
	ANCE	6.49	6.98	10.32	7.88	8.09	6.74	5.03	3.87	3.07	-	-
	QA-GNN	4.49	2.34	7.49	6.23	8.14	5.92	2.35	1.34	2.13	-	-
	HybGRAG	7.73	8.95	10.02	15.28	11.63	9.84	0.84	1.49	3.85	-	-
	ToG	5.33	3.31	5.54	10.29	8.39	10.02	0.53	2.81	1.77	-	-
	PullNet	4.08	5.31	6.46	7.24	4.83	4.37	1.15	1.43	1.81	-	-
	QE+LM (modular method)	0.24	0.20	0.33	0.32	0.53	0.58	0.0001	0.002	0.005	-	-
	HybridQE (BetaE)	11.18	10.93	14.40	21.75	13.36	13.33	1.52	3.18	6.68	-	-
	HybridQE (GammaE)	**14.65**	**15.9**	**20.15**	**28.61**	**17.94**	**15.05**	4.05	**4.57**	**8.94**	-	-
BioHybridQA	BM25	6.25	8.64	9.40	10.87	5.56	3.72	5.29	**5.25**	4.05	**6.04**	3.74
	ANCE	5.48	6.75	8.28	9.01	4.84	3.83	**5.35**	4.24	3.74	5.58	3.25
	QA-GNN	3.51	3.39	4.24	5.18	3.23	3.57	4.93	1.85	2.86	3.27	2.56
	HybGRAG	4.55	7.96	10.29	11.92	4.84	3.87	0.51	0.83	0.52	1.45	3.35
	ToG	0.74	1.23	0.27	0.78	1.02	2.06	0.59	0.47	0.49	0.48	0.05
	PullNet	2.90	4.17	5.02	5.73	3.36	2.74	0.73	0.91	1.16	3.13	2.01
	QE+LM (modular method)	0.37	0.50	0.67	0.59	0.43	0.75	0.003	0.05	0.05	0.31	0.42
	HybridQE (BetaE)	5.76	6.02	11.48	13.75	6.08	**4.85**	1.62	2.96	**4.68**	2.96	3.19
	HybridQE (GammaE)	**6.38**	**9.37**	**11.49**	**16.31**	**6.15**	4.11	1.58	2.79	4.58	3.55	**3.84**

minimize the following loss:

$$\text{Loss}(Q) ::= \sum_{a \in Q^G} \left(\frac{1}{|N_a|} \sum_{b \in N_a} \Delta_Q^b - \Delta_Q^a \right). \tag{10}$$

where $N_a \subseteq \neg Q^G$ is a set of negative samples for answer a regarding query Q.

Integration of Graph Information. Symbolic query embedding methods are trained on diverse queries from a training graph (a subset of the test graph), enabling strong generalization to unseen queries during inference. This stems from query diversity, which helps learning logical connectives, and structural integration via 1p queries. However, our datasets use hybrid 1p queries, which do not fully encode complete graph structure. To further improve HybridQE, we augment the training data by incorporating the entire training graph in the form of hybrid 1p queries with an empty text query component.

5 Experiments

In this section, we study the following research questions with experiments and analyses: **RQ1:** How well does HybridQE perform on predicting answers for hybrid queries over incomplete text-labeled graph databases? **RQ2:** Does hybrid query formulation improve reasoning over single-modality queries? **RQ3:** How robust is HybridQE when integrated with different query embedding methods? **RQ4:** How does the symbolic-to-text query ratio affect HybridQE's performance? Experimental details are supplemented in Appendix F.

5.1 Experimental Setup

Evaluation Metrics. We evaluate HybridQE on the task of hybrid query answering under incomplete data using the ProdHybridQA and BioHybridQA datasets (see Section 3), and compare the performance against baseline methods using the retrieval metrics MRR / Hits@1 / Hits@3.

Baselines. Since no prior work directly targets hybrid query answering over incomplete text-labeled graphs, we evaluate HybridQE against a collection of adapted baselines. These include LLM-enhanced graph RAG methods (ToG (Sun et al., 2024), HybGRAG (Lee et al., 2025)), classic and neural text retrieval systems (BM25 (Robertson and Zaragoza, 2009), ANCE (Xiong et al., 2021), QA-GNN (Yasunaga et al., 2021b)), QA method for semi-structured data (PullNet (Sun et al., 2019)) and a modular QE+LM approach that infers answers by combining symbolic query embedding predictions with LLM-based textual inference results. The specific introduction and implementation details on baselines are provided in Appendix F.2.

5.2 Analysis of Research Questions

Effectiveness of HybridQE (RQ1). Table 3 summarizes the MRR performance of HybridQE[4], instantiated with different query embedding backbones, alongside the adapted baselines on both

[4]More results in other metrics are available in Appendix I

Table 4: MRR Performance of HybridQE(GammaE) on Text-Only and Symbolic-Only questions from ProdHybridQA and BioHybridQA benchmark datasets.

Dataset	1p	2i	3i	2u	3u	1n	2in	3in	2p	3p
ProdHybridQA(Hybrid)	**15.9**	**20.15**	**28.61**	**17.94**	**15.05**	**4.05**	**4.57**	**8.94**	-	-
ProdHybridQA(Text Only)	8.18	15.83	16.57	13.63	11.35	2.34	1.35	4.02	-	-
ProdHybridQA(Symbolic Only)	10.29	10.61	14.42	11.58	8.64	2.73	2.54	7.25	-	-
BioHybridQA(Hybrid)	**9.37**	**11.49**	**16.31**	**6.15**	**4.11**	**1.58**	**2.79**	**4.58**	**3.55**	**3.84**
BioHybridQA(Text Only)	5.92	7.32	9.27	5.21	2.85	0.59	0.94	0.98	0.75	1.73
BioHybridQA(Symbolic Only)	6.38	5.49	7.48	2.35	2.46	0.32	1.37	0.96	1.32	1.34

benchmark datasets. Several observations can be drawn from these results.

Classic and neural retrieval baselines perform weakly across both datasets. BM25 and ANCE lack mechanisms to enforce logical constraints or compose multi-condition queries, while QA-GNN's graph-based propagation is severely hampered by missing edges in incomplete graphs, with textual signals insufficient to compensate. PullNet, despite its iterative KB-and-text retrieval design, underperforms even BM25 on most query types. In our incomplete graphs, missing edges cause subgraph expansion to terminate prematurely, preventing hard-answer entities from being reached. More critically, PullNet applies symbolic constraints only as post-hoc filters rather than integrating them during retrieval, confirming that logical constraints must be incorporated during, rather than after, the reasoning process. LLM-based baselines (ToG, HybGRAG) are limited by incomplete graph structure, which disrupts multi-hop reasoning chains, and by the inevitable loss of logical precision when hybrid queries are translated into natural language. Moreover, as shown in Appendix H, these methods exhibit high sensitivity to backbone choice with no consistent winner across query types. Modular method similarly fails, achieving only 0.24 average MRR on ProdHybridQA, nearly 60× lower than HybridQE(GammaE). Since QE+LM uses the same text encoder as HybridQE, this gap directly quantifies the value of joint hybrid reasoning over post-hoc score fusion.

HybridQE outperforms all baselines on average across both benchmarks, with HybridQE(GammaE) achieving 58.7% and 40.1% relative improvements over the strongest baseline (BM25) on ProdHybridQA and BioHybridQA respectively. Gains are most pronounced on structurally complex queries such as 3i and 2i, where cross-modal interaction provides the greatest benefit. We provide case analysis in Appendix J. For negation queries, performance remains lower, a known limitation of neural complement operators (Ren et al., 2023), though HybridQE(GammaE) still matches or outperforms the best baseline in most negation query types. Beyond the transductive setting, Appendix K demonstrates strong robustness in few-shot scenarios (17.36 average MRR for frequency-1 entities), highlighting HybridQE's potential for data-efficient and inductive hybrid reasoning.

Hybrid Query Formulation Impact (RQ2). To isolate the effect of hybrid query formulation, we compare the performance of HybridQE under three alternative representations of each test query: (i) **Text-Only** reformulates both the symbolic constraints and the textual condition into a single natural-language question, (ii) **Symbolic-Only** translates the entire query into a purely symbolic representation,[5] and (iii) **Hybrid** adopts our proposed formulation, which retains symbolic and textual constraints as distinct yet jointly processed inputs. The results are summarized in Table 4. We observe that HybridQE consistently achieves superior performance when queries are expressed in hybrid form compared to single-modal variants. This trend supports our hypothesis that jointly modeling symbolic and textual constraints is effective for querying semi-structured data.

HybridQE integrated with different query embeddings (RQ3). HybridQE is built upon query embedding techniques and is designed to maintain performance across backbone models. To verify its transferability across models, we evaluate it using BetaE (Ren and Leskovec, 2020) and GammaE (Yang et al., 2022), two query embedding methods capable of modeling $\mathcal{SROI}^-$ description concepts. Table 3 presents the performance of our method with BetaE and GammaE on two benchmark datasets. The results demonstrate that both models effectively support HybridQE in hybrid query answering across different graphs, con-

[5]The translation details are provided in Appendix G

firming its robustness. HybridQE (GammaE) outperforms HybridQE (BetaE), as GammaE exhibits superior reasoning capabilities, indicating that our method inherits and extends the reasoning ability of backbone models for hybrid query answering.

Influence of Symbolic-to-Text Query Ratio (RQ4). Hybrid queries differ in their composition, containing varying proportions of symbolic constraints and textual descriptions. A robust hybrid query answering approach should therefore adapt effectively to different levels of symbolic–textual integration. To assess this property, we analyze the performance of HybridQE under varying text-to-symbolic condition ratios. Specifically, we partition test queries into five groups based on the proportion of textual conditions: 0–20%, 20–40%, 40–60%, 60–80%, and 80–100%. 0% corresponds to purely symbolic queries, while a ratio of 100% corresponds to purely textual queries. As illustrated in Figure 3, HybridQE instantiated with GammaE exhibits stable and competitive performance across all composition ranges, demonstrating its robustness to varying degrees of symbolic and textual integration.

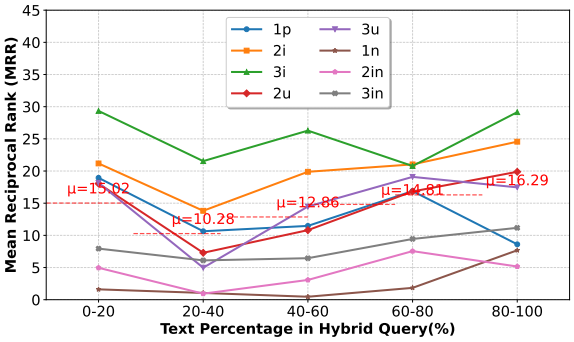


Figure 3: Performance of HybridQE (Gamma) on hybrid queries with varying text-to-symbolic ratio. μ represents the mean performance across different query types at each ratio.

6 Ablation Studies and Analysis

To assess the effectiveness of the components in HybridQE, we perform extensive ablation studies by considering several variant of HybridQE. We list HybridQE and its variants below: QCS with graph (our method): query-conditioned similarity with integration of graph information; QCS w/o graph: query-conditioned similarity only; MLP with graph: MLP network replacing query-conditioned similarity, retaining graph structure; and MLP w/o graph: MLP only. Figure 4 presents HybridQE's performance for its variants. Firstly,

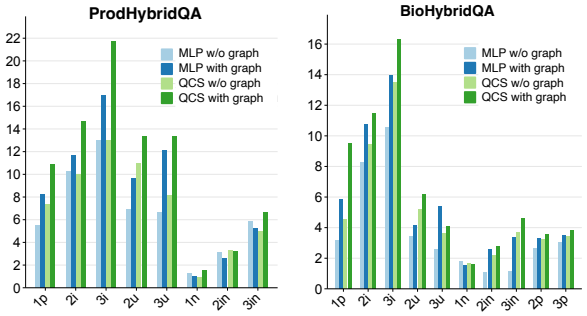


Figure 4: Ablation analysis on the performance of HybridQE(BetaE) with respect to model components.

the results show that incorporating graph structural information improves the performance, as graph-based variants consistently outperform those without. Secondly, the cross-attention mechanism further enhances performance across nearly all query types, further validating the effectiveness of our method's components in hybrid query reasoning.

7 Related Work

Early retrieval methods target individual modalities in isolation: text-based QA systems (Karpukhin et al., 2020; Lee et al., 2019) answer natural language questions over text, while QE methods (Ren et al., 2019; Ren and Leskovec, 2020; Yang et al., 2022; He et al., 2024) embed logical queries into a vector space for reasoning over incomplete knowledge graphs. Integrating both modalities, GraftNet and PullNet (Sun et al., 2018, 2019) retrieve jointly from KG triples and text passages, STaRK (Wu et al., 2025) benchmarks LLM-based retrieval over semi-structured knowledge bases with RAG and Graph RAG methods (Peng et al., 2024; Sun et al., 2024; Yasunaga et al., 2021a), and HybGRAG (Lee et al., 2025) combines text and relational retrieval in an agentic pipeline. However, none of these methods support formal logical operators over free-form textual descriptions, nor account for symbolic incompleteness, the two gaps our work addresses.

8 Conclusion

We introduce hybrid query answering over incomplete text-labeled graphs, a setting where logical precision and semantic flexibility can be jointly satisfied under missing data. Our hybrid query language, two new benchmarks, and HybridQE framework together demonstrate that unified symbolic-textual reasoning is both necessary and achievable, opening a concrete path toward expressive querying over the incomplete, semi-structured knowledge bases that dominate real-world applications.

Limitations

While our work demonstrates the effectiveness of hybrid query formulation and the proposed HybridQE framework, several limitations remain. First, our main experiments focus on the transductive setting, where entities and relations observed at test time appear during training; although we provide preliminary few-shot results, fully inductive hybrid query answering remains an open challenge. Second, HybridQE relies on the quality of pretrained text encoders to capture semantic information, which may limit performance in highly specialized domains or under noisy textual descriptions. Third, as observed in our experiments, modeling negation and complex logical operators continues to be challenging for neural query embedding methods, and further improvements in logical reasoning are needed.

References

Debayan Banerjee. 2023. Semantic parsing for knowledge graph question answering with large language models. In *The Semantic Web: ESWC 2023 Satellite Events: Hersonissos, Crete, Greece, May 28 - June 1, 2023, Proceedings*, page 234–243, Berlin, Heidelberg. Springer-Verlag.

Tengfei Feng and Liang He. 2025. RGR-KBQA: Generating logical forms for question answering using knowledge-graph-enhanced large language model. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 3057–3070, Abu Dhabi, UAE. Association for Computational Linguistics.

Cosimo Gregucci, Bo Xiong, Daniel Hernandez, Lorenzo Loconte, Pasquale Minervini, Steffen Staab, and Antonio Vergari. 2025. Is complex query answering really complex? *Preprint*, arXiv:2410.12537.

Ruining He and Julian McAuley. 2016. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *Proceedings of the 25th International Conference on World Wide Web*, WWW '16, page 507–517, Republic and Canton of Geneva, CHE. International World Wide Web Conferences Steering Committee.

Yunjie He, Daniel Hernández, Mojtaba Nayyeri, Bo Xiong, Yuqicheng Zhu, Evgeny Kharlamov, and Steffen Staab. 2024. Generating sroi¯ ontologies via knowledge graph query embedding learning. In *ECAI*, volume 392 of *Frontiers in Artificial Intelligence and Applications*, pages 4279–4286. IOS Press.

Bowen Jin, Yu Zhang, Yu Meng, and Jiawei Han. 2023. Edgeformers: Graph-empowered transformers for representation learning on textual-edge networks. In *ICLR*. OpenReview.net.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pages 6769–6781. Association for Computational Linguistics.

Haoyu Kuang, Jiarong Xu, Haozhe Zhang, Zuyu Zhao, Qi Zhang, Xuanjing Huang, and Zhongyu Wei. 2023. Unleashing the power of language models in text-attributed graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 8429–8441, Singapore. Association for Computational Linguistics.

Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. 2019. Latent retrieval for weakly supervised open domain question answering. In *ACL (1)*, pages 6086–6096. Association for Computational Linguistics.

Meng-Chieh Lee, Qi Zhu, Costas Mavromatis, Zhen Han, Soji Adeshina, Vassilis N. Ioannidis, Huzefa Rangwala, and Christos Faloutsos. 2025. Hybgrag: Hybrid retrieval-augmented generation on textual and relational knowledge bases. In *ACL (1)*, pages 879–893. Association for Computational Linguistics.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. Pytorch: An imperative style, high-performance deep learning library. In *NeurIPS*, pages 8024–8035.

Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. *CoRR*, abs/2408.08921.

Hongyu Ren, Mikhail Galkin, Michael Cochez, Zhaocheng Zhu, and Jure Leskovec. 2023. Neural graph reasoning: Complex logical query answering meets graph databases. *Preprint*, arXiv:2303.14617.

Hongyu Ren, Weihua Hu, and Jure Leskovec. 2019. Query2box: Reasoning over knowledge graphs in vector space using box embeddings. In *ICLR*.

Hongyu Ren and Jure Leskovec. 2020. Beta embeddings for multi-hop logical reasoning in knowledge graphs. *NeurIPS*, 33:19716–19726.

Stephen E. Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Found. Trends Inf. Retr.*, 3(4):333–389.

Tao Shen, Xiubo Geng, Guodong Long, Jing Jiang, Chengqi Zhang, and Daxin Jiang. 2020. Effective search of logical forms for weakly supervised

knowledge-based question answering. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, pages 2227–2233. International Joint Conferences on Artificial Intelligence Organization. Main track.

Haitian Sun, Tania Bedrax-Weiss, and William W. Cohen. 2019. Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, pages 2380–2390. Association for Computational Linguistics.

Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William W. Cohen. 2018. Open domain question answering using early fusion of knowledge bases and text. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 4231–4242. Association for Computational Linguistics.

Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *ICLR*. OpenReview.net.

Shirley Wu, Shiyu Zhao, Michihiro Yasunaga, Kexin Huang, Kaidi Cao, Qian Huang, Vassiis N. Ioannidis, Karthik Suhhian, James Zou, and Jure Leskovec. 2025. Stark: benchmarking llm retrieval on textual and relational knowledge bases. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA. Curran Associates Inc.

Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *ICLR*. OpenReview.net.

Huimin Xu, Wenting Wang, Xin Mao, Xinyu Jiang, and Man Lan. 2019a. Scaling up open tagging from tens to thousands: Comprehension empowered attribute value extraction from product title. In *ACL (1)*, pages 5214–5223. Association for Computational Linguistics.

Huimin Xu, Wenting Wang, Xin Mao, Xinyu Jiang, and Man Lan. 2019b. Scaling up open tagging from tens to thousands: Comprehension empowered attribute value extraction from product title. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5214–5223, Florence, Italy. Association for Computational Linguistics.

Yao Xu, Shizhu He, Jiabei Chen, Zihao Wang, Yangqiu Song, Hanghang Tong, Kang Liu, and Jun Zhao. 2024. Generate-on-graph: Treat LLM as both agent and KG in incomplete knowledge graph question answering. *CoRR*, abs/2404.14741.

Hao Yan, Chaozhuo Li, Ruosong Long, Chao Yan, Jianan Zhao, Wenwen Zhuang, Jun Yin, Peiyan Zhang, Weihao Han, Hao Sun, Weiwei Deng, Qi Zhang, Lichao Sun, Xing Xie, and Senzhang Wang. 2023. A comprehensive study on text-attributed graphs: Benchmarking and rethinking. In *Advances in Neural Information Processing Systems*, volume 36, pages 17238–17264. Curran Associates, Inc.

Dong Yang, Peijun Qing, Yang Li, Haonan Lu, and Xiaodong Lin. 2022. Gammae: Gamma embeddings for logical queries on knowledge graphs. *EMNLP*.

Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021a. QA-GNN: reasoning with language models and knowledge graphs for question answering. In *NAACL-HLT*, pages 535–546. Association for Computational Linguistics.

Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021b. QA-GNN: reasoning with language models and knowledge graphs for question answering. In *NAACL-HLT*, pages 535–546. Association for Computational Linguistics.

Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. In *ACL (2)*. The Association for Computer Linguistics.

Xinyang Zhang, Chenwei Zhang, Xian Li, Xin Luna Dong, Jingbo Shang, Christos Faloutsos, and Jiawei Han. 2022. Oa-mine: Open-world attribute mining for e-commerce products with weak supervision. In *Proceedings of the ACM Web Conference 2022*, WWW '22, page 3153–3161, New York, NY, USA. Association for Computing Machinery.

Jianan Zhao, Meng Qu, Chaozhuo Li, Hao Yan, Qian Liu, Rui Li, Xing Xie, and Jian Tang. 2023. Learning on large-scale text-attributed graphs via variational inference. In *ICLR*. OpenReview.net.

Ruilin Zhao, Feng Zhao, and Hong Zhang. 2025. Correcting on graph: Faithful semantic parsing over knowledge graphs with large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 5364–5376, Vienna, Austria. Association for Computational Linguistics.

Jason Zhu, Yanling Cui, Yuming Liu, Hao Sun, Xue Li, Markus Pelger, Tianqi Yang, Liangjie Zhang, Ruofei Zhang, and Huasha Zhao. 2021. Textgnn: Improving text encoder via graph neural network in sponsored search. In *WWW*, pages 2848–2857. ACM / IW3C2.

A Syntax and Semantics of $\mathcal{SROI}^-$ concept

Definition 3 (Syntax of $\mathcal{SROI}^-$ concept descriptions)**.** *Given a knowledge graph $G = (V, R, E)$, $\mathcal{SROI}^-$ concept descriptions C are generated by the grammar:*

$$C ::= \top \mid \{a\} \mid \neg C \mid C \sqcap C \mid \exists r.C \mid \exists r^-.C\,,$$

where $\top$ is the universal concept, $a \in V$, and $r \in R$. Nominals $\{a\}$ denote singleton sets. As abbreviations, we write $\bot = \neg\top$, $C \sqcup D = \neg(\neg C \sqcap \neg D)$, and $\forall r.C = \neg\exists r.\neg C$.

Definition 4 (Semantics of $\mathcal{SROI}^-$ concept)**.** *The interpretation of C on graph G is defined recursively as:*

1. $\top^G = V$,
2. $\{a\}^G = \{a\}$,
3. $\neg C^G = V \setminus C^G$,
4. $C \sqcap D^G = C^G \cap D^G$,
5. $\exists r.C^G = \{u \mid \exists (u, r, v) \in E,\ v \in C^G\}$,
6. $\exists r^-.C^G = \{u \mid \exists (v, r, u) \in E,\ v \in C^G\}$.

For example, the concept ∃type.Coffee retrieves all items of type coffee (i.e., a, b), while ∃flavor.Roast⊓∃brand.¬Starbucks retrieves roast-flavored items not branded Starbucks (i.e., only a).

B Generation of Datasets

In this paper, we construct two new benchmark datasets, BioHybridQA and ProdHybridQA, for the hybrid query answering task over incomplete text-labeled graph. This section provides a detailed description of the procedure used to construct hybrid queries for our benchmark datasets. The goal of the generation process is to produce queries that (i) contain both symbolic and textual components that enforce logical constraints and capture semantic nuances respectively, and (ii) require reasoning under symbolic incompleteness.

B.1 Main Algorithm

In Algorithm 1, we present the psudo code for the generation of hybrid query answering datasets.[6] Table 5 summarizes the statistics on the generated queries.

Algorithm 1: Hybrid Query Generation

Input:
- Knowledge Graph $G = (V, R, E)$, where:
- V: set of entities
- R: set of relations
- E: set of triples $(h, r, t) \in V \times R \times V$, $E = E_{test}$ where $E_{train} \subset E_{valid} \subset E_{test}$
- Ent2Text, Rel2Text
- query_structure $\in Q_{\text{templates}}$ for different query structures

Output: Hybrid query $Q = [\Phi, q]$ where Φ is symbolic query, q is textual query and query answer set A

Initialize: $A_{easy}, A_{hard} \leftarrow \emptyset, \emptyset$ `// Answer set`

while $A_{hard} = \emptyset$ **do**

  Step 1: Entity Sampling

    $e_{\text{target}} \leftarrow$ SampleEntity(V) or SampleEntity(I) if hyperedge graph `// Randomly sample an entity as an answer entity`

  Step 2: Symbolic Query Generation

    $\Phi \leftarrow$ GenSymbolicQuery$(e_{\text{target}}, \text{query_structure}, G)$ `// Generate a symbolic query following the query structure`

  Step 3: Triple Selection for Textual Query

    $E_{\text{relevant}} \leftarrow \{(h, r, t) \in E_{test} \mid t = e_{\text{target}}\}$

    $E_{\text{selected}} \leftarrow$ SelectRandomSubset(E_{relevant}, k) `// Select random` k `triples involving` e_{target}

  Step 4: Textual Query Generation

    $q_{\text{sym}} \leftarrow$ TriplesToQuery(E_{selected})

    $q \leftarrow$ QueryToNLQuestion$(q_{\text{sym}}, \text{Ent2Text}, \text{Rel2Text})$ `// Convert triples to semantically equivalent natural language question about` e_{target}

  Step5: Retrieve Answers for the Hybrid Query

    $A_{\text{easy}} \leftarrow \{a \mid G_{\text{train}} \models q_{\text{sym}}(a) \wedge \Phi(a)\}$

    $A_{\text{hard}} \leftarrow \{a \mid G_{\text{test}} \models q_{\text{sym}}(a) \wedge \Phi(a)\}$ - A_{easy} `// Derive the answer set involving with triples missing from train graph`

end

Return $Q = [\Phi, q]$, A_{hard} `// The hybrid query and the hard answer set`

B.2 Fulfilling Dataset Assumptions

We now systematically justify that the generation procedure satisfies both dataset assumptions: (i). Queries contain both symbolic and textual components that enforce logical constraints and capture semantic nuances respectively. (ii) Queries require reasoning under symbolic incompleteness.

Assumption I. The symbolic component Φ, generated by GenSymbolicQuery, encodes formal logical operators — conjunction, disjunction,

[6] Since code is used to generate queries for triple graphs, we adapt it on hyperedge graphs by composing nodes and hyperedge indexes into entities, and transforming hyperedges into triples of the form (hyperedge index, relation, node), where hyperedge indexes = I and nodes = N, thus $V = \{I, N\}$.

Table 5: Descriptive statistics for ProdHybridQA (Amazon) and BioHybridQA (Prime). ProdHybridQA is a hyperedge product graph and does not support multi-hop projection queries (2p, 3p), indicated by —.

		ProdHybridQA		**BioHybridQA**	
Category	**Statistic**	**Value**	**Detail**	**Value**	**Detail**
Graph	Product / bio entities	86,905	prod nodes	129,385	bio nodes
	Attribute entities	36,037	attr nodes	—	—
	Total entities	122,942		129,385	
	Relation types	16	attr rels	37	bio rels
	Training edges	424,476		13,064,296	
Splits	Train queries	113,682	8 structures	289,483	10 structures
	Valid queries	23,233	8 structures	58,476	10 structures
	Test queries	23,295	8 structures	58,539	10 structures
Per type		**train / valid / test**		**train / valid / test**	
	1p	13,402 / 2,748 / 2,797		26,740 / 5,441 / 5,405	
	2p	—		29,328 / 5,872 / 5,854	
	3p	—		29,526 / 5,914 / 5,906	
	2i	14,605 / 2,959 / 2,969		29,107 / 5,880 / 5,862	
	3i	14,963 / 2,997 / 2,998		29,863 / 5,975 / 5,970	
	2u	14,605 / 2,963 / 2,970		29,107 / 5,891 / 5,875	
	3u	14,963 / 2,997 / 2,999		29,863 / 5,978 / 5,982	
	1n	12,509 / 2,703 / 2,706		27,557 / 5,728 / 5,770	
	2in	13,956 / 2,894 / 2,885		28,752 / 5,835 / 5,921	
	3in	14,679 / 2,972 / 2,971		29,640 / 5,962 / 5,994	

negation, and multi-hop projection — as a valid $\mathcal{SROI}^-$ concept description (He et al., 2024) that cannot be approximated by natural language without precision loss. The textual component q is derived from triples *disjoint* from those used to instantiate Φ, ensuring q captures complementary semantic information rather than restating symbolic constraints. The semantic abstraction applied in QUERYTONLQUESTION of Step 4 employs large language models (with instructions given below) to ensure that q expresses nuanced, qualitative descriptions, such as inferring *"lightweight"* from a weight attribute value, or *"suitable for beginners"* from a style attribute that cannot be recovered by symbolic lookup or string matching.

Assumption II. Symbolic incompleteness is enforced through the construction of the hard answer set $A_{\text{hard}}(Q) = \{a \mid G_{\text{test}} \models q_{\text{sym}}(a) \wedge \Phi(a)\} \setminus A_{\text{easy}}$. By definition, every answer in $A_{\text{hard}}(Q)$ satisfies the hybrid query over the complete graph G_{complete} but *not* over the observed training graph G_{train}. The **while** loop in Algorithm 1 guarantees that every generated query has a non-empty hard answer set ($A_{\text{hard}} \neq \emptyset$), ensuring that all queries in our benchmarks strictly require inference over missing information. The missing information may be recoverable via two complementary pathways that our framework is designed to exploit. The *structural pathway* leverages latent relational patterns in G_{train} to infer missing edges, the mechanism exploited by symbolic query embedding methods (Ren and Leskovec, 2020; Yang et al., 2022). The *textual pathway* leverages the entity text descriptions $\tau(v)$ to infer missing attributes from semantic evidence, for example, inferring that an entity satisfies a weight constraint from a description mentioning "lightweight design", even when the weight triple is absent from G_{train}.

Prompt for Semantic Abstraction

Instruction. You are a natural language generation expert. Given a structured attribute description of the form `[relation]: [value]`, rephrase it into a semantically equivalent but lexically varied natural language phrase. Apply semantic abstraction where appropriate:

- *Convert specific measurements to qualitative descriptors, e.g.,* `weight: 275g` → "a lightweight racket"
- *Map categorical values to conceptual descriptions, e.g.,* `style_fit: Tennis Beginner` → "suitable for beginner players"
- *Express relational attributes implicitly, e.g.,* `brand: HEAD` → "a HEAD-manufactured product"

Do NOT alter the meaning, omit information, or introduce facts not present in the input. Output only the rephrased phrase with no explanation.

Input: `[relation]: [value]`
Output: `[rephrased phrase]`

B.3 Lexical Overlap Analysis.

A potential concern with triple-derived textual queries is that they may merely paraphrase sym-

bolic graph attributes, reducing the task to surface-level string matching. To assess this, we measure the token-level F1 score between each textual query q and the raw attribute values of its target entity e_{target} in the graph. Specifically, token-level F1 is computed after lowercasing, punctuation removal, and stopword filtering, and is taken as the maximum F1 across all entity attributes to give the most lenient possible overlap estimate. For example, given the textual query q = *"Find a lightweight tennis racket made by HEAD"* and entity attributes {`brand` : HEAD, `weight` : 275g, `category` : Tennis Racket}, the query tokens after filtering are {*lightweight, tennis, racket, made, HEAD*} and the attribute tokens for `brand` are {*HEAD*}, for `weight` are {*275g*}, and for `category` are {*tennis, racket*}. The per-attribute F1 scores are therefore 0.33, 0.00, and 0.57 respectively.

Figure 5a shows the distribution of token-level F1 scores between textual query components and raw graph attribute values across both datasets. The distributions are strongly right-skewed, with the bulk of queries concentrated below F1 $= 0.4$, confirming that the majority of generated queries do not directly copy graph attribute values. The average token-level F1 is 0.34 on ProdHybridQA and 0.30 on BioHybridQA, both well below the near-verbatim threshold of 0.8. Critically, only 3.03% and 2.55% of queries in ProdHybridQA and BioHybridQA respectively exceed this threshold, indicating that near-verbatim paraphrase is rare by construction. Figure 5b further confirms that no individual query type is trivially recoverable by string matching. Together, these results confirm that our benchmarks require genuine semantic reasoning and cannot be solved by paraphrase recovery.

C Specific details of neural logical operators within HybridQE

In this section, we supplement the specific details of the operators, i.e., relational transformation, intersection operator, union operator and complement operator, of the query embedding models involved in our experiments.

C.1 BetaE

BetaE represents concepts by the Cartesian product of multiple Beta distributions: $\mathbf{Emb}_C = [(\alpha_1, \beta_1), \ldots, (\alpha_n, \beta_n)]$ where each component is a Beta distribution $\text{Beta}(\alpha, \beta)$ controlled with two shape parameters α and β.

- **Relational Transformation** maps from one Beta embedding $\mathbf{S}$ to another Beta embedding $\mathbf{S}'$ given the relation type r. This is modeled by a transformation neural network for each relation type r using a multi-layer perceptron (MLP):

$$\text{RelT}_r(\mathbf{Emb}_C) = \text{MLP}_r(\mathbf{Emb}_C) \quad (11)$$

- **Intersection Operator** is modeled by taking the weighted product of the PDFs of the input Beta embeddings

$$I = \frac{1}{Z} \prod_{i=1}^{n} p_{\mathbf{Emb}_C}^{w_i} \quad (12)$$

where $I = \text{Intersect}(\mathbf{Emb}_{C_1}, \cdots, \mathbf{Emb}_{C_n})$ and Z is a normalization constant and $w_1, \cdots, w_n$ are the weights with their sum equal to 1.

- **Complement Operator** is modeled by taking the reciprocal of the shape parameters.

$$C = \left[\left(\frac{1}{\alpha_1}, \frac{1}{\beta_1}\right), \ldots, \left(\frac{1}{\alpha_n}, \frac{1}{\beta_n}\right)\right] \quad (13)$$

where $C = \text{Complement}(\mathbf{Emb}_C)$.

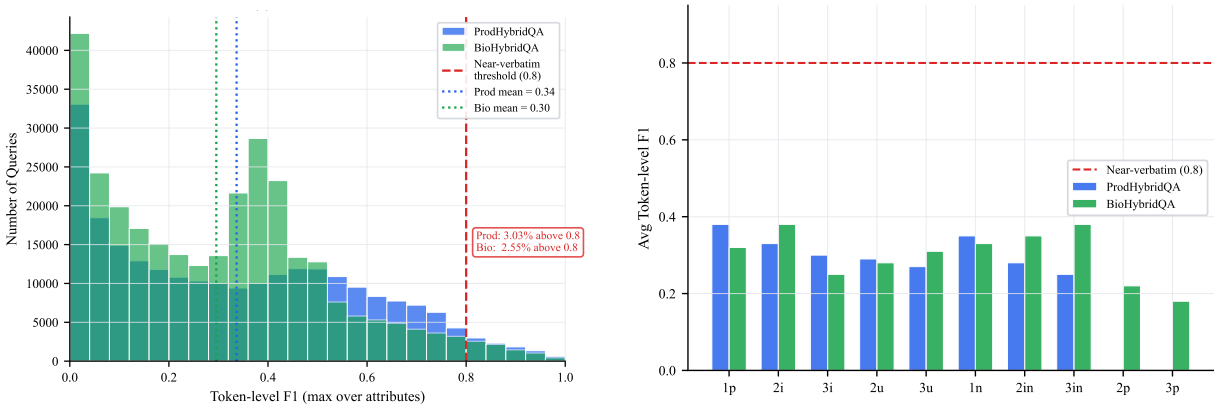
- **Union Operator** BetaE does not model the specific union operator because queries involving with union can be translated into disjunctive normal form (DNF), more details can be checked in (Ren and Leskovec, 2020).

- **Distance function** Given an answer entity embedding $\mathbf{a}$ with parameters $[(\alpha_1^a, \beta_1^a), \ldots, (\alpha_n^a, \beta_n^a)]$, and a query embedding $\mathbf{q}$ with parameters $[(\alpha_1^q, \beta_1^q), \ldots, (\alpha_n^q, \beta_n^q)]$, we define the distance between this entity a and the query q as the sum of KL divergence between the two Beta embeddings along each dimension:

$$\text{dist}_{\text{beta}}(a; q) = \sum_{i=1}^{n} \text{KL}\left(p_{\mathbf{a},\mathbf{i}}; p_{\mathbf{q},\mathbf{i}}\right) \quad (14)$$

C.2 GammaE

GammaE models concepts by the Cartesian product of multiple Gamma distributions: $\mathbf{Emb}_C = [(\alpha_1, \beta_1), \ldots, (\alpha_n, \beta_n)]$ where each component is a Gamma distribution $\Gamma(\alpha, \beta)$ controlled with shape parameter α and rate parameter β.



(a) Distribution of F1 scores

(b) Avg F1 per Query Type

Figure 5: Lexical Overlap Analysis between Textual Queries and Graph Attribute Values.

- **Relational Transformation** maps from one Gamma embedding $\mathbf{S}$ to another Gamma embedding $\mathbf{S}'$ given the relation type r. This is modeled by a transformation neural network for each relation type r using a multi-layer perceptron (MLP):

$$\text{RelT}_r(\mathbf{Emb}_C) = \text{MLP}_r(\mathbf{Emb}_C) \quad (15)$$

- **Intersection Operator** is modeled by taking the weighted product of the PDFs of the input Gamma embeddings

$$I = \frac{1}{Z}\prod p^{w_1}_{\mathbf{Emb}_C} \cdots p^{w_n}_{\mathbf{Emb}_C} \quad (16)$$

where $I = \text{Intersect}(\mathbf{Emb}_{C_1}, \cdots, \mathbf{Emb}_{C_n})$ and Z is a normalization constant and $w_1, \cdots, w_n$ are the weights with their sum equal to 1.

- **Negation Operator** is modeled by taking the reciprocal of the shape parameters α along with an eslasticity pamater ϵ, such that

$$C = [\Gamma(\frac{1}{\alpha_1}, \beta_1) + \epsilon, \ldots, \Gamma(\frac{1}{\alpha_n}, \beta_n) + \epsilon] \quad (17)$$

where $C = \text{Complement}(\mathbf{Emb}_C)$.

- **Union Operator** is modeled by a Gamma mixture models, it is calculated as a weighted sum of Gamma distributions, such that

$$U = \sum_{i=1}^{n} \theta_i \Gamma(\alpha_i, \beta_i) \quad (18)$$

where $U = \text{Union}(\mathbf{Emb}_{C_1}, \cdots, \mathbf{Emb}_{C_n})$ and $\theta_i = \frac{\exp(\Gamma(\alpha_i, \beta_i))}{\sum_{j=1}^{n}\exp(\Gamma(\alpha_j, \beta_j))}$ is the learned weight for each Gamma density in the Gamma mixture model and also uses the self-attention mechanism.

- **Distance function** Given an answer entity embedding $\mathbf{a}$ with parameters $[(\alpha^a_1, \beta^a_1), \ldots, (\alpha^a_n, \beta^a_n)]$, and a query embedding $\mathbf{q}$ with parameters $[(\alpha^q_1, \beta^q_1), \ldots, (\alpha^q_n, \beta^q_n)]$, we define the distance between this entity a and the query q as the sum of KL divergence between the two Gamma embeddings along each dimension:

$$\text{dist}_{\text{Gamma}}(a; q) = \sum_{i=1}^{n} \text{KL}\left(\Gamma_{\mathbf{a,i}}; \Gamma_{\mathbf{q,i}}\right) \quad (19)$$

D Query Types considered in Dataset ProdHybridQA

Figure 6 illustrates the query types considered in the dataset ProdHybridQA. Being different from triple graph which support multi-hop queries, hyperedge graphs do not support 2p, 3p queries.

E Examples of Queries from Dataset BioHybridQA

Table 6 presents the query examples from the dataset BioHybridQA.

F Experiment Details

F.1 Hyperparameters and Computational Resource

All of our experiments are implemented in Pytorch (Paszke et al., 2019) framework and run on four

Table 6: BioHybridQA Query Examples

Query Type	Hybrid Query Example
1p	[expression_present(urinary bladder), "Which gene or protein shows expression in both the endometrium and the myometrium?"]
2p	[target(ppi(PRKCG)), "Which drug targets the GABRA1 gene or protein?"]
3p	[target(associated_with(ppi(BCL2L14)), "Which biological process interacts with RPEL1 and is part of the parent–child relationship within the glyoxylate cycle?"]
2i	[interacts_with^{-}(actin filament polymerization)∧expression_present^{-}(tibial nerve), "Which gene or protein has a protein–protein interaction (PPI) with HTR3A?"]
3i	[expression_present(corpus callosum)∧interacts_with(endoplasmic reticulum membrane)∧ expression_present(esophagus), "Which gene or protein interacts with the apoptotic cleavage of cellular proteins and is associated with CADDS?"]
1n	[¬interact_with(protein binding), "Which gene or protein shows no expression in the quadriceps femoris?"]
2in	[¬expression_present(heart left ventricle)∧type(gene/protein), "Which gene or protein is expressed in the lymph node?"]
3in	[¬synergistic_interaction(demeclocycline)∧type(drug)∧synergistic_interaction(quinidine), "Which drug has a synergistic interaction with magnesium glycinate?"]
2u	[indication(systemic mastocytosis)∨synergistic_interaction(aldosterone), "Which drug is associated with the side effect of pulmonary fibrosis?"]
3u	[interacts_with(protein binding)∨expression_present(temporal lobe)∨type(gene/protein), "Which gene or protein has a protein–protein interaction with PEX5 and also interacts with the endocytic recycling process?"]

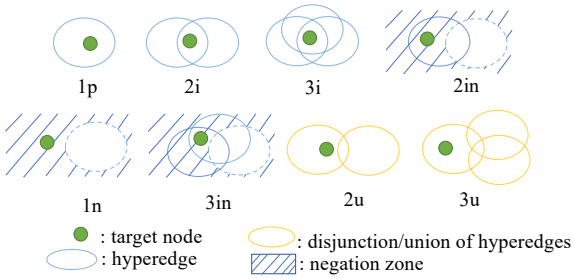


Figure 6: Query structures of the symbolic queries considered in the ProdHybridQA dataset, where anchor entities and relations are to be specified to instantiate logical queries. Each structure is named using abbreviations: "p" for projection of a hyperedge, "i" for intersection, "u" for union, and "n" for negation. For instance, "2i" indicates the conjunction of two atomic queries.

Nvidia A100 GPU cards. Most LLM-based baselines are implemented with OpenAI API (GPT3.5-turbo for the backbone of ToG and HybGRAG) and Ollama (Deepseek-R1-Distill-Qwen-32B and Deepseek-R1-Distill-Llama-8B). For hyperparameters search, we performed a grid search of learning rates in $\{5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}\}$, the batch size in $\{256, 512, 1024\}$, the negative sample sizes in $\{128, 64\}$, and the margin γ in $\{30, 40, 50, 60, 80\}$. The best hyperparameters are shown in Table 8. The code of HybridQE and the created benchmark datasets can be accessed at https://anonymous.4open.science/r/HybridQE-BC3C/README.md.

F.2 Introduction and Implementation of Baselines

We introduce the technical details on the implementation of the baseline methods.

BM25, ANCE and QA-GNN We adapt the baseline methods on the hybrid query answering task by transforming the hybrid queries into natural language questions under the textualization templates in Appendix G.2 and treating the entity descriptions as the search corpus. The implementations of the baseline methods are following the open-sourced code: BM25 (https://github.com/nhirakawa/BM25/tree/master), ANCE (https://github.com/microsoft/ANCE), and QAGNN(https://github.com/michiyasunaga/qagnn).

ToG on HybridQA We adapt ToG on the hybrid query reasoning task through the following steps, respecting its original design:

1. **Question Transformation**: transforming the hybrid queries into natural language questions under the textualization templates in Appendix G.2

2. **Initialization**: extract topic entities from the input query, the topic entities are symbolic query entities in our implementation

Table 7: The Hits@1 performance of HybridQE and baseline models on ProdHybridQA and BioHybridQA benchmark datasets.

Dataset	Model	1p	2i	3i	2u	3u	1n	2in	3in	2p	3p
ProdHybridQA	BM25	3.33	5.66	6.66	6.33	8.17	3	1	0.67	-	-
	ANCE	2.94	3.52	2.59	4.29	2.62	1.42	0.32	0.23	-	-
	QA-GNN	0.38	2.45	2.27	3.02	2.94	2.01	0.17	0.19	-	-
	HybGRAG	2.73	4.69	7.95	5.24	5.38	0.39	0.94	0.48	-	-
	ToG	1.38	2.19	3.59	3.75	8.03	0.14	0.97	0.54	-	-
	PullNet	2.15	2.83	3.57	1.91	1.73	0.35	0.42	0.56	-	-
	QE+LM (modular method)	0.11	0.21	0.16	0.23	0.25	0	0	0	-	-
	HybridQE (BetaE)	6.57	7.44	10.06	8.24	8.36	2.1	1.16	2.15	-	-
	HybridQE (GammaE)	**9.48**	**12.20**	**16.21**	**10.23**	**13.25**	**3.59**	**1.29**	**2.77**	-	-
BioHybridQA	BM25	1.66	3.33	5.00	**1.66**	2.33	**1.66**	1.66	1.00	**3.00**	0.33
	ANCE	2.23	2.11	3.14	1.38	1.72	1.32	0.95	0.94	2.87	0.84
	QA-GNN	1.07	1.94	2.39	1.02	1.94	1.1	0.78	0.78	2.05	0.95
	HybGRAG	2.31	2.92	2.58	1.79	2.94	0.29	0.18	0.17	0.98	**1.57**
	ToG	0.68	0.11	0.14	0.38	1.01	0.12	0.13	0.09	0.13	0.00
	PullNet	1.52	2.15	2.63	1.26	1.53	0.22	0.35	0.32	1.11	0.62
	QE+LM (modular method)	0.23	0.28	0.21	0.18	0.31	0.00	0.002	0.013	0.12	0.14
	HybridQE (BetaE)	2.68	2.35	7.67	1.93	3.73	1.01	**2.18**	**2.52**	1.45	1.22
	HybridQE (GammaE)	**3.63**	**5.01**	**7.94**	**1.71**	**2.71**	0.92	1.75	2.51	1.01	1.07

Table 8: Hyperparameters found by grid search. d is the embedding dimension, b is the batch size, n is the negative sampling size, γ is the margin in loss, l is the learning rate, ω is the regularization parameter in the distance function.

Dataset	Model	d	b	n	γ	l
ProdHybridQA	HybridQE (BetaE)	800	512	128	50	1e-4
	HybridQE (GammaE)	800	512	128	60	1e-4
BioHybridQA	HybridQE (BetaE)	800	512	128	50	1e-4
	HybridQE (GammaE)	800	512	128	60	1e-4

3. **Iterative Exploration (Beam Search)**: we iteratively perform relation exploration and entity exploration. For **relation exploration**, we retrieve candidate relations linked to the current entities from the KG and use the LLM to prune irrelevant relations, keeping the top-N most relevant ones. For **entity exploration**, we retrieve candidate entities linked by the selected relations and prune entities via the LLM, retaining the most promising ones. Finally, we **extend reasoning paths** with the selected relations and entities.

4. **Reasoning and Termination Check**: After each exploration step, the LLM evaluates whether the current paths contain enough information to answer the question.

5. **Answer Generation**: the LLM synthesizes the final answer using the explored paths or falls back to its internal knowledge if paths are insufficient.

HybGRAG on HybridQA HybGRAG is an agentic hybrid retrieval framework designed for hybrid question answering over semi-structured knowledge bases that contain both textual documents and relational graph structures. To adapt HybGRAG to our hybrid query answering setting, we reimplement its core components following the original design.

1. **Query routing and entity–relation extraction**: Given a translated natural language query, an LLM-based router first identifies the relational aspect of the query by extracting candidate topic entities and relevant relation types from a predefined schema. Based on the extracted information, the router decides whether to invoke a text retrieval module or a hybrid (graph-based) retrieval module.

2. **Retriever bank**: HybGRAG maintains a retriever bank consisting of two modules. The text retrieval module performs dense vector similarity search between the translated question and textual documents (entity descriptions). The hybrid retrieval module first extracts an ego-graph around the identified entities using the selected relations, and then ranks the associated documents using text similarity, enabling joint use of relational constraints and textual relevance.

3. **Critic module with self-reflection**: To address errors in initial routing and extraction, HybGRAG employs a critic module composed of a validator and a commenter. The validator checks whether the retrieved documents satisfy the query requirements, while

the commenter provides structured corrective feedback when retrieval fails. This feedback is used to refine entity and relation extraction in subsequent iterations.

4. **Iterative refinement**: The router, retriever bank, and critic operate in an iterative loop. The process terminates when the validator accepts the retrieved results or when a maximum number of refinement steps is reached.

In our experiments, we follow this procedure to adapt HybGRAG as a baseline for hybrid query answering, while keeping its retrieval-centric and agentic design unchanged.

QE+LM on HybridQA QE+LM answers hybrid queries by independently processing symbolic and textual constraints and combining their outputs. To ensure a fair comparison with HybridQE, we use the same text encoder for the textual module. The pipeline consists of three stages:

1. **Symbolic Query Processing (QE Module)**: we retrained BetaE on the training graph to embed the symbolic query into latent space and retrieve entities from the KG whose embeddings minimize distance to the query embedding, yielding answer set A_{sym}.

2. **Textual Query Processing (LM Module)**: We compute embeddings for the textual query component and all entity descriptions using the same sentence-transformer encoder as HybridQE. Then, entities are ranked based on semantic similarity (e.g., cosine distance) between query and entity embeddings. Top-10 entities with highest similarity scores, yielding answer set A_{text}.

3. **Answer Fusion**: we derive the final answer set as the conjunction of A_{sym} and A_{text}, which ensures answers satisfying both structural (KG) and semantic (textual) constraints.

The key distinction between QE+LM and HybridQE is therefore architectural: QE+LM processes the two modalities in isolation and combines outputs post-hoc, while HybridQE enables bidirectional information flow between symbolic and textual components during encoding. Any performance difference thus reflects the value of joint reasoning rather than encoder quality.

PullNet on HybridQA PullNet is an iterative retrieval framework that constructs a question-specific subgraph by alternately pulling facts from a KB and text from a corpus, then reasoning over the resulting heterogeneous graph with a graph CNN to extract answers. We adapt PullNet to HybridQA as follows.

1. **Query input.** Hybrid queries are converted to natural language questions using the textualization templates in Appendix 10.

2. **Graph mapping.** Entity nodes correspond to graph entities V, fact nodes to training edges E_{train}, and text nodes to the textual descriptions $\tau(v)$ attached to each entity, replacing PullNet's external corpus with node-attached text descriptions.

3. **Initialisation.** The initial subgraph is seeded with the symbolic anchor entities extracted directly from ϕ, bypassing entity linking.

4. **Relation scoring.** The pull operation scores candidate relations using a weighted combination of symbolic relevance (whether the relation appears in ϕ) and textual similarity to the query's text component t, controlled by a hyperparameter λ.

5. **Answer ranking.** Since HybridQA requires a ranked list of entities, we score all entities by their graph CNN classifier score if reached by iterative retrieval, and by text similarity to the query's textual component otherwise. Logical constraints in ϕ are applied as post-hoc filters over the ranked list. This modular design contrasts with HybridQE's joint encoding, and serves to isolate the contribution of integrated hybrid reasoning.

G Translation of Hybrid Queries

G.1 Transform Hybrid Queries into Symbolic Form

In this section, we describe the text-to-logic compilation step that parses the free-form text into an approximate symbolic constraint, then conjunct it with the original symbolic part in the query. Given the hybrid query Q = $\{t\} \sqcap \phi$, a LLM-based parser $\mathcal{P}_{\text{LLM}}$, and a schema linking function $\mathcal{L}$, the transformation process consists of three steps:

Table 9: The Hits@3 performance of HybridQE and baseline models on ProdHybridQA and BioHybridQA benchmark datasets.

Dataset	Model	1p	2i	3i	2u	3u	1n	2in	3in	2p	3p
ProdHybridQA	BM25	8.33	14.66	11.33	13.66	10.33	**5.33**	**4.66**	3.00	-	-
	ANCE	5.29	10.85	**5.33**	8.22	5.05	3.94	3.92	3.14	-	-
	QA-GNN	1.42	6.27	4.29	7.54	4.25	1.38	0.92	1.32	-	-
	HybGRAG	5.94	6.49	12.85	9.59	7.43	0.71	1.13	2.73	-	-
	ToG	2.35	4.24	8.48	7.39	9.32	0.34	2.15	1.23	-	-
	PullNet	5.83	7.15	8.32	5.41	4.95	1.22	1.64	2.17	-	-
	QE+LM (modular method)	0.42	0.44	0.51	0.67	0.54	0.00	0.00	0.002	-	-
	HybridQE (BetaE)	7.04	11.35	20.01	10.59	8.12	1.45	3.01	5.83	-	-
	HybridQE (GammaE)	**11.76**	**17.79**	**25.57**	**15.44**	**13.18**	1.54	3.59	**6.53**	-	-
BioHybridQA	BM25	13.01	12.00	12.33	8.33	7.33	**5.66**	4.33	4.23	**7.33**	8.33
	ANCE	6.49	10.56	10.53	6.84	**7.49**	5.59	3.03	3.69	2.35	0.95
	QA-GNN	5.54	7.59	8.34	6.83	1.59	1.36	2.37	2.81	2.48	0.92
	HybGRAG	4.85	2.93	2.28	2.43	2.35	1.86	0.39	0.45	2.35	2.23
	ToG	2.48	2.92	2.27	1.47	2.04	1.19	1.34	0.15	0.21	0.01
	PullNet	4.64	5.83	6.75	3.93	3.26	0.85	1.52	1.32	3.63	2.31
	QE+LM (modular method)	1.87	1.38	1.94	1.38	1.28	0.002	0.01	0.02	0.39	0.92
	HybridQE (BetaE)	10.82	13.02	15.95	7.47	6.43	2.37	3.39	3.69	3.43	6.24
	HybridQE (GammaE)	**16.67**	**16.61**	**20.72**	**9.81**	7.42	2.71	**4.83**	**4.28**	5.65	**8.72**

1. **Parsing**: We prompt a language model with task-specific instructions to translate natural language queries into conjunctive queries composed of abstract predicates and variables, which are later grounded to the database schema via similarity-based linking:

$$\hat{\phi}_t = \mathcal{P}_{\text{LLM}}(t)$$

where $\hat{\phi}_t$ represents the symbolic format of the textual query. The prompt instructions for query parsing are provided below.

2. **Grounding**: Since the predicates and entities in $\hat{\phi}_t$ may not directly correspond to schema elements in the target text-labeled graph database, we perform a grounding step that aligns abstract symbols with concrete nodes and relations. For each entity and relation symbol in $\hat{\phi}_t$, we compute its textual similarity to candidate nodes and relation names in the database, and select the best-matching schema elements. This produces a grounded conjunctive query ϕ_t whose predicates and arguments are defined over the database schema:

$$\phi_t = \mathcal{L}(G, \hat{\phi}_t)$$

3. **Composition**: The grounded conjunctive query ϕ_t is then conjoined with the original symbolic constraints ϕ to form the final compiled symbolic query, which serves as input to downstream hybrid reasoning and retrieval:

$$Q_{symbolic} = \phi_t \sqcap \phi$$

Prompt for Text-to-Conjunctive Query Parsing

Instruction. You are an expert in semantic parsing and knowledge graph querying. Given a natural language query, translate it into a conjunctive query (CQ) that captures the logical constraints expressed in the text. Use only conjunction ($\wedge$), disjunction ($\vee$), and negation ($\neg$) when necessary. Introduce variables for entities whose identity is unknown. Do not ground predicates or entities to a specific database schema; use descriptive predicate and entity names inferred from the text.
Output format. Represent the query as a set of atoms of the form `predicate(subject, object)`. Use variables prefixed with ?. Output only the conjunctive query.

G.2 Transform Hybrid Queries into Textual Form

We transform the symbolic component in hybrid queries into natural language questions using domain-specific templates that maintain the original logical operations while producing fluent questions. Table 11 lists the query structures of queries presented in our datasets. Table 10 presents the query-to-question templates for datasets ProdHybridQA and BioHybridQA. The templates systematically handle conjunctions, disjunctions, and negations while preserving the original query structure.

H Backbone Sensitivity of LLM-based Methods

A natural question is whether stronger LLM backbones improve performance on HybridQA. To investigate this, we evaluate ToG with three backbone models of varying capability: GPT-3.5-turbo, DeepSeek-R1-Distill-Qwen-32B, and DeepSeek-R1-Distill-Llama-8B. The results are reported in

Table 10: Query-to-Question Templates

Query Type	Template
ProdHybridQA Queries	
Base	"Please find out the products with the following attributes:"
1p/1n	+ "[relation] is/is not [entity]"
2i/2in	+ "[relation1] is [entity1], and [relation2] is/is not [entity2]"
3i/3in	+ "[relation1] is [entity1], [relation2] is [entity2], and [relation3] is/is not [entity3]"
2u	+ "[relation1] is [entity1], or [relation2] is [entity2]"
3u	+ "[relation1] is [entity1], or [relation2] is [entity2], or [relation3] is [entity3]"
BioHybridQA Queries	
Base	"Please find out items that"
1p	+"they [relation] [entity]"
1n	+"they do not [relation] [entity]"
2p	+"they [relation1] something that [relation2] [entity]"
3p	+"they [relation1] something that [relation2] something that [relation3] [entity]"
2i	+"they [relation1] [entity1] and [relation2] [entity2]"
3i	+"they [relation1] [entity1], [relation2] [entity2] and [relation3] [entity3]"
2in	+"they [relation1] [entity1] but do not [relation2] [entity2]"
3in	+"they [relation1] [entity1] and [relation2] [entity2] but do not [relation3] [entity3]"
2u	+"they [relation1] [entity1] or [relation2] [entity2]"
3u	+"they [relation1] [entity1] or [relation2] [entity2] or [relation3] [entity3]"

Table 11: Query structure templates of query types in HybridQA datasets, where e_i corresponds to entity i and r_i corresponds to relation i.

Query Type	Query Structure
1p	$(e_1, (r_1))$
1n	$(e_1, (r_1, n))$
2p	$(e_1, (r_1, r_2))$
3p	$(e_1, (r_1, r_2, r_3))$
2i	$((e_1, (r_1)), (e_2, (r_2)))$
3i	$((e_1, (r_1)), (e_2, (r_2)), (e_3, (r_3)))$
2in	$((e_1, (r_1)), (e_2, (r_2, n)))$
3in	$((e_1, (r_1)), (e_2, (r_2)), (e_3, (r_3, n)))$
2u	$((e_1, (r_1)), (e_2, (r_2)), u)$
3u	$((e_1, (r_1)), (e_2, (r_2)), (e_3, (r_3)), u)$

Figure 7.

The results indicate that LLM-based methods exhibit **high sensitivity to backbone choice**. The performance varies substantially across backbone models, with no consistent winner across query types or datasets. This instability contrasts sharply with HybridQE, which maintains consistent improvements across all query types regardless of its symbolic backbone (BetaE or GammaE), as shown in RQ3.

I More Experimental Results

In this section, we provide more experimental results of HybridQE and baseline approaches in different evaluation metrics, Hits@1 and Hits@3, in Table 7 and Table 9 respectively.

J Qualitative Case Study

To complement the quantitative results in Section 5, we present a qualitative analysis of a representative case of HybridQE. The goal is to more vividly illuminate how HybridQE reasons at intermediate steps.

Query. The 3i hybrid query

$$Q = \phi \sqcap \{t\}$$

where the symbolic and textual components are defined as:

$$\phi = \exists \textit{category}.\{\textit{Baby_Bottle}\} \sqcap \exists \textit{material_type}.\{\textit{BPA free}\} \sqcap \textit{color}.\{\textit{Pink}\}$$

$$t = \text{"}\textit{Could you help locate a bottle categorized as a baby product with a capacity measured in milliliters?}\text{"}$$

Rank comparison. Table 12 presents an internal decomposition of HybridQE's scoring on the hybrid query $Q = \phi \sqcap \{t\}$. For each candidate entity we report three ranks derived from a single HybridQE forward pass: the rank induced by the symbolic component score $\text{Dist}(\mathbf{Emb}_\phi, \mathbf{Emb}_e)$ alone, the rank induced by the textual component score $\text{Dist}(\mathbf{Emb}_{\{t\}}, \mathbf{Emb}_e)$ alone, and the final rank produced by the full hybrid encoder $\text{Dist}(\mathbf{Emb}_Q, \mathbf{Emb}_e)$ after the INTERSECT operator combines both components. Note that this

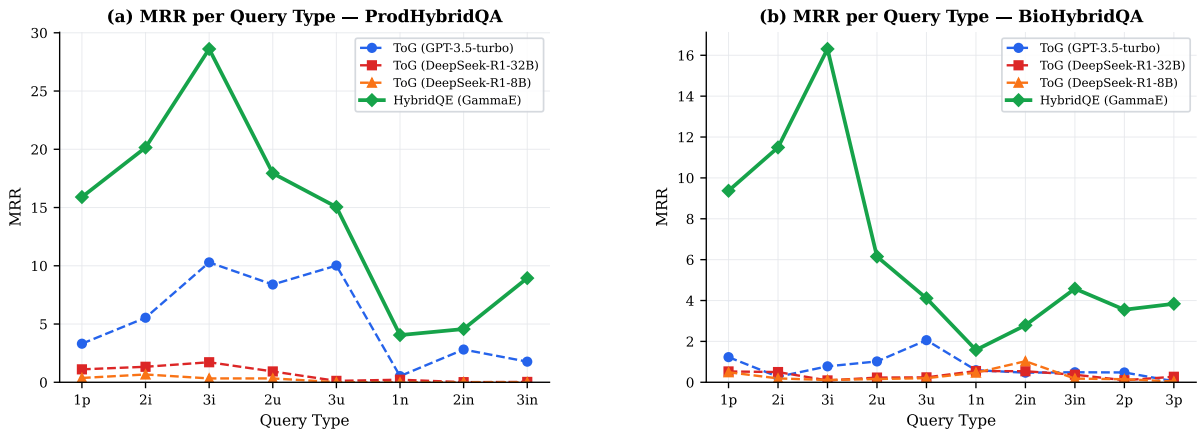


Figure 7: Backbone sensitivity analysis of ToG with three LLM backbones compared to HybridQE (GammaE).

Table 12: Internal rank decomposition of HybridQE on the 3i success-case hybrid query Q. For each entity we report the rank induced by the symbolic component $\mathbf{Emb}_\phi$, the textual component $\mathbf{Emb}_{\{t\}}$, and the final hybrid encoder $\mathbf{Emb}_Q$, all derived from the same single forward pass on $Q = \phi \sqcap \{t\}$. $\downarrow$ lower rank is better.

Entity	$\mathbf{Emb}_\phi \downarrow$	$\mathbf{Emb}_{\{t\}} \downarrow$	$\mathbf{Emb}_Q \downarrow$	**Description**
B078TGQHQQ†	2	4	**1**	Putti Atti Silicone Baby Bottle with Handle, BPA Free, Pink
B07RR2BRHD	5	3	2	Tescoma Papu Papi Soft Bottle with Spoon, 200 ml, Pink
B00DBFXG1Q	3	8	3	BellyBottle Pregnancy Water Bottle Intake Tracker with Weekly Milestone Stickers, Pregnancy Gifts for First Time Moms, Must-Have Essentials, Pink
B07YN985ZV	4	10	4	Baby Magic Bottles (2 Pack) Milk / Orange Juice — Empties When Tilted, with 2 GosuToys Stickers
B00IM3QW7C	12	6	5	Tommee Tippee Closer to Nature Baby Bottle Decorated Pink, Breast-Like Nipple for Natural Latch, Slow Flow, BPA-Free - 0+ Months, 2 Count
B01MRKFA85	7	12	6	Baby Bottle Adapter Turns Water Bottles into Baby Bottles, Formula or Bottled Water for Babies On-The-Go by Refresh-A-Baby, Camo Green

† Correct hard-answer entity ($\in \mathcal{A}_{\text{hard}}$).

decomposition is distinct from the query-level ablation in RQ2 (Table 3), which runs HybridQE on differently-formulated queries; here we inspect the intermediate scores produced by the two component encoders *within a single hybrid query forward pass*.

Analysis. Table 12 provides direct empirical evidence for HybridQE's two complementary inference pathways (Section 4), The key finding concerns the correct hard-answer entity B078TGQHQQ, whose symbolic edges all belong to $E_{missing}$. Despite this, $\mathbf{Emb}_\phi$ assigns it a strong rank 2, reflecting the structure-driven pathway: latent relational patterns in G_{train} provide partial structural evidence even for entities with missing edges. The *textual pathway* further reinforces this recovery: B078TGQHQQ's description explicitly contains *"BPA Free"* and *"Pink"*, the exact values of the two missing *material_type* and *color* edges in ϕ, allowing $\mathbf{Emb}_{\{t\}}$ to recover the missing symbolic information directly from text.

Additionally, the remaining entities illustrate why neither component alone suffices. B07RR2BRHD ranks higher than others under the textual component, but its weak symbolic rank 5 leads the hybrid encoder to correctly demote it to rank 2. B00DBFXG1Q achieves a strong symbolic rank 3 yet receives rank 8 from text encoder for lacking any capacity signal, and is correspondingly demoted to rank 3 by the hybrid encoder. Across all entities, the hybrid encoder consistently promotes those with mutually reinforcing evidence from both modalities and penalizes those strong on one axis

but weak on the other, confirming that joint encoding enables cross-modal verification that neither modality alone can achieve.

K Few-shot Performance Analysis

Different from backbone symbolic query embedding methods, HybridQE is capable of handling hybrid query answering in inductive setting. Although our benchmark datasets are transductive, we examine HybridQE in few-shot scenarios to gain insights into its potential robustness on unseen entities—a key aspect of inductive settings. Figure 8 illustrates the distribution of entity mention frequencies within the training dataset. There are more than 10,000 entities being mentioned only few times in the train dataset. We reclassified the test queries according to the frequency with which the entities referenced in the queries appear in the training dataset. Table 13 presents the performance of HybridQE on test queries grounded with different levels. Our results demonstrate that HybridQE maintains robust performance on one-shot queries, indicating its effectiveness in few-shot scenarios. However, the observed degradation in performance on over-grounded queries suggests that the model may overfit to frequent patterns, thereby compromising its generalization capability on these queries.

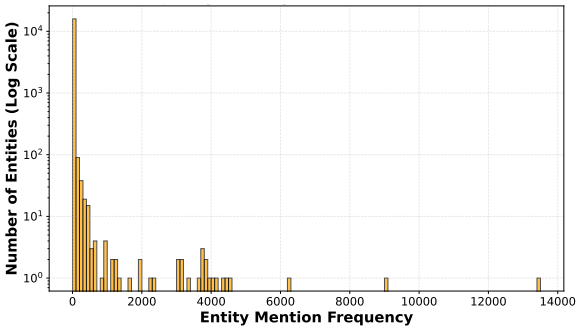


Figure 8: Distribution of entity mention frequencies within the training dataset. The x-axis denotes the frequency of entity mentions, while the y-axis corresponds to the number of entities associated with each frequency value on log scale.

Table 13: The MRR performance of HybridQE on test queries grounded with varying frequency in the training graph.

Frequency	**1p**	**2i**	**3i**	**2u**	**3u**	**1n**	**2in**	**3in**	**AVG**
1	34.65	28.41	36.3	10.67	14.77	1.12	4.89	8.07	17.36
2-50	35.55	45.42	36.89	18.58	19.3	0.99	8.58	13.87	22.40
50-2000	12.19	19.12	17.61	12.58	9.14	0.55	0.51	8.57	10.03
$\geq$ 2000	3.67	13.79	2.26	9.49	12.59	1.71	1.97	6.25	6.47